
Multi-Head Attention Residuals

Cheng Luo*

Independent Researcher
wdlctc@gmail.com

Zefan Cai*

University of Wisconsin–Madison
zefncai@gmail.com

Junjie Hu

University of Wisconsin–Madison
junjie.hu@wisc.edu

Abstract

Transformers propagate information across depth through a single additive residual stream: every sublayer reads only the most recent state. *Attention residuals* relax this by letting each sublayer attend, through a learned softmax. However, that read uses a *single* query shared across the entire width, so every feature subspace must read the depth history through one distribution. The cost of this *forced compromise* grows with how much the subspaces disagree about which layers to read, and disagreement grows with model width. We introduce **Multi-Head Attention Residuals (MHAR)**: the routing query is reshaped into H per-subspace heads, each with its own softmax over the depth history. The read becomes block-diagonal, the reshape adds *zero* parameters and negligible compute, and $H=1$ recovers attention residuals exactly. Trained from scratch on a deduplicated Nemotron-based anneal corpus that is quality-filtered and STEM- and code-heavy, MHAR improves validation loss over a standard Transformer at 100M, 350M, and 1B (-0.061 , -0.149 , and -0.140). It achieves the best result among four methods in every setting, with the gain increasing from 100M to the larger scales. The head count is a real design axis rather than a free knob: validation loss is U-shaped with respect to H , with a flat optimum at $H=4$ or $H=8$ across scales. We adopt $H=8$ for large-scale models; over-splitting beyond this point ($H=16$) consistently gives back part of the gain. A direct probe of the trained queries confirms that learned subspace disagreement is the underlying driver. Fused Triton routing kernels increase attention-residual training throughput from $0.2\text{--}0.5\times$ to $0.55\text{--}0.88\times$ of the baseline while maintaining near-baseline peak memory. An identity-preserving conversion using *delta* attention residuals supports 8B mid-training, yielding improvements of $+3.2$ on GSM8K and $+3.1$ on GPQA.

1 Introduction

Transformers [Vaswani et al., 2017] propagate information across depth through a single residual stream [He et al., 2016]: layer ℓ reads $\mathbf{h}_{\ell-1}$ and writes $\mathbf{h}_\ell = \mathbf{h}_{\ell-1} + f_\ell(\mathbf{h}_{\ell-1})$. The running sum couples every sublayer’s input to the immediately preceding output, even though useful features may have been computed many layers earlier. *Attention residuals* relax this by treating the set of all prior sublayer outputs as a memory and letting each sublayer retrieve a learned, softmax-weighted combination of them, an attention over depth rather than over tokens [Kimi, 2025].

This retrieval makes the depth history addressable, but only at the granularity of the *entire width*. A single learned query $\mathbf{q} \in \mathbb{R}^d$ [Kimi, 2025] scores the $N=2L+1$ prior sources and collapses them into one softmax distribution α over depth; all d coordinates of the retrieved mixture are then read through that same α : every feature dimension is forced to read from the same layers in the same proportions (Figure 1, zoom left).

Structurally, this is a single self-attention query that routes over the depth history rather than the token history. Self-attention is multi-head precisely because one query cannot serve every subspace

*Equal contribution. Corresponding authors: wdlctc@gmail.com, zefncai@gmail.com, junjie.hu@wisc.edu.

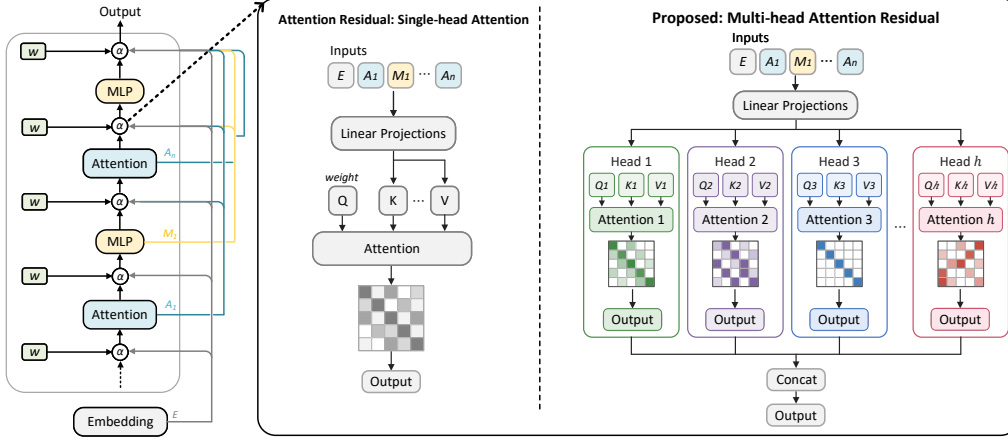


Figure 1: **The depth read is an attention, so it should be multi-head.** **Left:** attention residuals [Kimi, 2025] feed every sublayer a learned mixture of the depth history (embedding and every earlier attention/MLP output). **Zoom left:** one routing site is exactly *single-head* attention over that history—one query, one depth map shared by all d channels (Eq. 2). **Zoom right:** MHAR gives each of H subspaces its own map (Eq. 3), parameter-free; $H=1$ recovers attention residuals exactly.

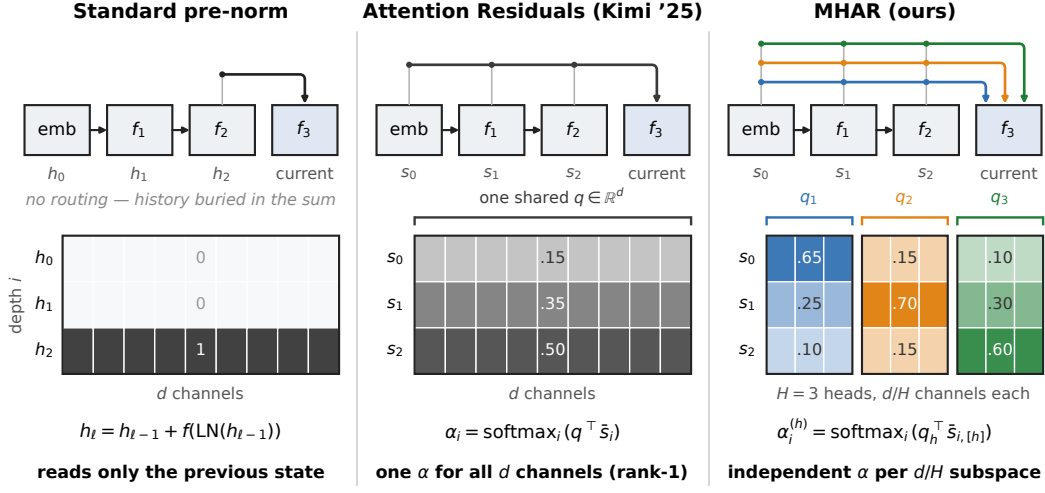
at once: the token read is already split into heads; the depth read is not. The restriction is not benign: one shared distribution collapses the differing depth preferences of every subspace into a single read.

We therefore propose **Multi-Head Attention Residuals (MHAR)**, a strict generalization that splits the routing query into H per-subspace heads, each routing independently over the depth history, and recovers attention residuals exactly at $H=1$ (Figure 1, zoom right). The change is parameter-free (the single $(d,)$ query is reshaped to $(H, d/H)$) and adds negligible compute. The mechanism’s real cost is at the systems level: every sublayer re-reads the depth history, and reference implementations pay that traffic several times over. We therefore also provide fused Triton routing kernels that remove most of this overhead (Appendix E), making depth routing practical at scale.

We frame this restriction as a concrete cost, the *forced compromise*: a single query must reconcile, in one softmax, the differing depth preferences of every subspace. Its cost is driven primarily by how much those subspaces disagree, which grows with model width; a secondary, more speculative factor is how collinear (hard to tell apart) the candidate sources are. Splitting the query into per-subspace heads makes the read block-diagonal and removes the compromise. We confirm the width dependence both in loss (§3) and by directly probing the trained queries (Appendix D).

Our contributions:

- **Multi-head attention residuals (MHAR).** We split the single attention-residual routing query into H per-subspace heads (§2): a reshape that adds no parameters or FLOPs, recovers attention residuals exactly at $H=1$, and needs little tuning—the optimum consistently lands at $H=4$ or $H=8$ across scales [Ainslie et al., 2023], and for large-scale models we adopt $H=8$ (Appendix C).
- **Fused kernels for efficient MHAR.** Fused Triton routing kernels lift attention-residual training from $\sim 0.2\text{--}0.5\times$ to $0.55\text{--}0.88\times$ baseline throughput at near-baseline peak memory (Appendix E), making depth routing practical at scale.
- **Scaling MHAR.** Trained from scratch at 100M–1B, MHAR is the best of four methods at every scale, with gains over the baseline that *grow* from 100M to the larger scales ($-0.061/-0.149/-0.140$; §3), and the routing-head count shows a clear U-shaped optimum at $H=4$ or $H=8$ (we adopt $H=8$ at scale; §4). An identity-preserving conversion [Luo et al., 2026] extends MHAR to 8B mid-training, adding $+3.2$ GSM8K (paired $p=0.004$) over a schedule-matched control (§3.3).



$W[i, c]$: read weight of depth row i into channel c of the current input (darker = larger); $\tilde{s}_i = \text{RMSNorm}(s_i)$

MHAR = same query reshaped $(d,) \rightarrow (H, d/H)$ — 0 new params, same FLOPs; $H = 1$ recovers Attn. Residuals

Figure 2: **What each method lets the current sublayer read across depth.** Each column: forward stack (top) and the depth-read weight matrix (bottom; entry $[i, c]$ is how strongly channel c reads depth row i ; illustrative). **Standard pre-norm**: a one-hot read of the previous state. **Attention residuals**: one shared query reads all earlier sources—every column identical (rank-1). **MHAR (ours)**: each subspace reads depth independently (block-wise columns).

2 Method: Multi-Head Attention Residuals

2.1 Background: the residual stream as depth memory

A pre-norm decoder Transformer [Vaswani et al., 2017, Xiong et al., 2020] of width d with L blocks maintains a single running state and updates it additively (Figure 2, left). Writing the two sublayers of block ℓ (attention and MLP) as f_ℓ and g_ℓ ,

$$\mathbf{h}'_\ell = \mathbf{h}_{\ell-1} + f_\ell(\text{LN}(\mathbf{h}_{\ell-1})), \quad \mathbf{h}_\ell = \mathbf{h}'_\ell + g_\ell(\text{LN}(\mathbf{h}'_\ell)). \quad (1)$$

Because the state is a running sum, every sublayer reads only the most recent state $\mathbf{h}_{\ell-1}$. Features computed many layers earlier are still present in the sum, but are no longer individually addressable: the network cannot selectively re-read the output of, say, layer 3 when computing layer 30 without that information first surviving every intervening additive update.

2.2 Attention residuals

Attention residuals [Kimi, 2025] make past sublayer outputs individually addressable by treating them as a memory and *routing* over them. Let the network expose the ordered list of *sources* $\mathcal{S} = (s_0, s_1, \dots, s_{N-1})$ of $N=2L+1$ sources, where s_0 is the token embedding and each later source is the raw output of one attention or MLP sublayer. In place of the additive update (Eq. 1), the input to a sublayer is a learned, softmax-weighted combination of *all* sources produced so far. With a per-sublayer learned pseudo-query $\mathbf{q} \in \mathbb{R}^d$ and an RMSNorm applied to the sources as keys,

$$\alpha_i = \frac{\exp(\mathbf{q}^\top \text{RMSNorm}(s_i))}{\sum_{j=0}^M \exp(\mathbf{q}^\top \text{RMSNorm}(s_j))}, \quad \tilde{\mathbf{h}} = \sum_{i=0}^M \alpha_i s_i, \quad (2)$$

where M indexes the sources available at that point. This is an attention over *depth* (the source/layer axis) rather than over tokens; the analogue of keys are the normalized sources and the analogue of values are the sources themselves, with a single learned query per routing site (Figure 2, middle).

```

1 def route(self, sources: list[Tensor], proj: Linear, norm: RMSNorm):
2     """Multi-head depth routing: split the (D,) query into self.H heads
3     and run H independent softmaxes over the sources. This kernel is
4     the ONLY change from attention residuals (Kimi 2025)."""
5     V = torch.stack(sources) # [N, B, T, D]
6     N, B, T, D = V.shape
7     q = proj.weight.view(self.H, D // self.H) # (D,) query -> H heads
8     Vh = V.view(N, B, T, self.H, D // self.H) # H slices per source
9     logits = einsum('h e, n b t h e -> n b t h', q, norm(Vh))
10    alpha = logits.softmax(0) # H softmaxes over depth
11    mixed = einsum('n b t h, n b t h e -> b t h e', alpha, Vh)
12    return mixed.reshape(B, T, D)
13
14 def forward(self, sources): # IDENTICAL to attention residuals (Kimi 2025)
15     # --- Attention sublayer ---
16     h = self.route(sources, self.attn_proj, self.attn_norm)
17     sources.append(self.attn(norm(h))) # append output as a new source
18
19     # --- MLP sublayer ---
20     h = self.route(sources, self.mlp_proj, self.mlp_norm)
21     sources.append(self.mlp(norm(h))) # replacement: append, don't add
22
23     return sources

```

Figure 3: **Multi-Head Attention Residuals pseudocode.** The forward pass is *identical* to attention residuals [Kimi, 2025]: each sublayer reads a routed combination of the prior sources and *appends* its raw output as a new source (replacement routing). The *sole* change is the `route` kernel—it splits the single $(d,)$ query into self.H heads and runs H independent softmaxes over depth, adding *zero* parameters. Substituting the single-head kernel ($H=1$) recovers attention residuals exactly (Appendix B).

2.3 Multi-head routing

A single query (Eq. 2) forces *all* d coordinates of the routed mixture to share one distribution α over depth: every feature dimension must read from the same layers in the same proportions. Different feature subspaces, however, may be best served by different layers. We remove this constraint with **multi-head routing (MHAR)** (Figure 2, right): partition the coordinates into H contiguous heads of width d/H , give each head its own query, and let each head route independently. For head h with query $\mathbf{q}_h \in \mathbb{R}^{d/H}$ and writing $\mathbf{x}_{[h]}$ for the h -th slice of a vector $\mathbf{x}$,

$$\alpha_i^{(h)} = \text{softmax}_i\left(\mathbf{q}_h^\top \text{RMSNorm}(\mathbf{s}_i)_{[h]}\right), \quad \tilde{\mathbf{h}}_{[h]} = \sum_i \alpha_i^{(h)} \mathbf{s}_{i,[h]}, \quad (3)$$

and the H routed slices are concatenated to width d . Each head scores only its own slice of each source and mixes only that slice, so the H softmaxes are fully independent and different subspaces can attend to different layers. Figure 3 shows the implementation: the forward pass is *identical* to attention residuals [Kimi, 2025]; the *only* change is the `route` kernel (the single-head original is recovered at $H=1$, Appendix B).

Properties. (i) *Parameter-matched.* The H queries $\{\mathbf{q}_h\}_{h=1}^H$ together form an $(H, d/H)$ tensor with exactly d parameters—the same as the single $(d,)$ query. Multi-head routing therefore adds *zero* parameters over single-head routing; H is a reshape, not a widening. (ii) *FLOP-matched.* The scoring and mixing in Eq. 3 touch each source coordinate exactly once, as in Eq. 2; the only overhead is computing H small softmaxes over the depth axis (length $\leq 2L+1$) instead of one, which is negligible next to the attention and MLP sublayers. (iii) *Strict generalization of attention residuals.* At $H=1$ the route reduces to the single shared query of Eq. 2 run through the identical forward, so the *single-head* method in our tables is the original attention-residual formulation of Kimi [2025] (Appendix B)—making MHAR-vs-single-head a strict iso-parameter, iso-forward control.

Fused routing kernels. Depth routing is memory-bound: scoring and mixing (Eq. 3) perform only a constant number of operations per source element read, with none of the data reuse that

lets the attention and MLP matrix multiplies run compute-bound, so its cost is the bytes it moves, and every sublayer re-reads the whole depth history ($O(L^2 B T d)$ traffic per step). A reference implementation pays this traffic several times over and retains the per-sublayer stacked sources, the dominant activation cost. We provide fused, deterministic Triton forward/backward kernels that cut per-microbatch routing time by $2\times$ over `torch.compile` and $6.4\times$ over the eager reference ($\sim 30\%$ end-to-end mid-training throughput on $8\times H100$). Appendix E gives the full forward/backward algorithm, the delta variant used for mid-training, and numerical verification.

3 Experiments

3.1 Setup

Models and training. We train decoder-only Transformers from scratch on the `anneal_pt_v3` corpus—a deduplicated, Nemotron-based anneal mix [NVIDIA, 2025b] (quality-filtered, synthetic-, STEM-, and code-heavy), the same corpus used for the 8B mid-training experiments of §3.3 (composition in Appendix Table 13)—for 20K steps with the AdamW optimizer [Loshchilov and Hutter, 2019], a cosine learning-rate schedule with 1,000 warmup steps decaying to $0.1\times$ peak, and `bfloat16`. (A replication of the same four-method comparison on a second, web-generic corpus is reported in Appendix Table 10.) The three scales are: 100M ($d=512$, $L=12$, 8 heads, 4 KV heads, FFN 1536), 350M ($d=1024$, $L=24$, 16 heads, 8 KV heads, FFN 4096), and 1B ($d=1280$, $L=36$, 16 heads, 8 KV heads, FFN 5120), all with head dimension 64–80 and a Qwen3-style architecture [Qwen, 2025]. The 100M models use sequence length 2048 and global batch 64; the 350M/1B models use sequence length 1024 and global batch 32. We compare four methods at each scale: a standard Transformer (*baseline*), hyper-connections [Zhu et al., 2024], single-head attention residuals ($H=1$, i.e. the published attention-residual formulation of Kimi [2025]), and multi-head attention residuals (*MHAR*) with the number of routing heads H set equal to the number of KV heads. Following Kimi [2025], all routing pseudo-queries are *zero-initialized*, so every depth softmax starts as a uniform average over its sources (the appendix replication predates this fix and uses randomly initialized queries; see the caveat there). All four methods share architecture, data, schedule, optimizer, and data-parallel degree, so the routing mechanism is the only difference. Table 1 reports all methods at a unified peak learning rate of 5×10^{-4} ; a per-method learning-rate sweep (Appendix Table 11) shows the method ranking is not an artifact of sharing one rate. MHAR is parameter-matched to baseline up to the $O(d)$ routing queries and adds only 0.5–1.2% routing FLOPs (Table 4). Full training and reproducibility details (optimizer hyperparameters, tokenizer, positional scheme, precision, and the validation protocol) are given in Appendix I.

Metric. A single validation pass is noisy at 1B ($\sim \pm 0.07$ over an evaluation pass; Appendix D), so unless noted we report the *tail-mean* of the last eleven evaluations (steps 15–20k). This ± 0.07 is single-pass sampling noise; the cross-method Δ is far better resolved, because methods are compared *paired* (same data order within a scale) and averaged over eleven evals. Pairing matters doubly here: each streaming validation pass samples a small window of the corpus, and long documents make per-eval values fluctuate across steps, so only same-step paired deltas and tail-means are meaningful. The single-cell tables report one fixed seed (Appendix I).

3.2 Pretraining

Table 1: From-scratch validation loss ($\downarrow$) after a unified 20K-step schedule on the `anneal_pt_v3` corpus at a unified peak learning rate of 5×10^{-4} (tail-mean over the last quarter of training; §3); Δ is relative to the same-rate baseline. MHAR is best at every scale, and both attention-residual variants *grow* their gain from 100M to the larger scales. The last column is an *over-splitting control*: the same model with the routing query split into $H=16$ heads, past the $H=4$ –8 optimum. A replication on a second corpus, where the single-head column reverses, is in Appendix Table 10.

Scale	$d/L/KV$	Baseline	Hyper-conn.	Single-head	MHAR ($H=8$)	MHAR ($H=16$)
100M	512/12/4	3.031	2.999 (−0.032)	2.970 (−0.060)	2.969 (−0.061)	2.996 (−0.035)
350M	1024/24/8	2.997	2.931 (−0.066)	2.876 (−0.121)	2.848 (−0.149)	2.895 (−0.102)
1B	1280/36/8	2.894	2.807 (−0.086)	2.759 (−0.134)	2.754 (−0.140)	2.825 (−0.069)

Comparison to hyper-connections. The most relevant learned alternative to the additive residual is hyper-connections [Zhu et al., 2024], which replace the single stream with n parallel streams mixed by per-layer learned depth and width connections. Trained in the same sweep ($n=4$), they improve over the baseline at all three scales ($-0.032/-0.066/-0.086$; Table 1), but **MHAR outperforms hyper-connections at every scale** (by 0.029, 0.083, and 0.053 at 100M/350M/1B): enriching the depth *routing* of a single stream is consistently stronger than enriching the *connection topology* across streams, at a fraction of the parameter and memory-traffic cost (Table 4).

Over-splitting control ($H=16$). The head count is bounded on both sides. The last column of Table 1 repeats MHAR with the same $(d,)$ query reshaped into $H=16$ heads—*past* the $H=4-8$ optimum ($2\times$ the KV count at 350M/1B, $4\times$ at 100M), every other setting identical. At 350M this gives back a third of MHAR’s gain ($-0.149 \rightarrow -0.102$) and at 1B half of it ($-0.140 \rightarrow -0.069$); at 100M the penalty is present but shallower. At all three scales $H=16$ lands *above* single-head routing. The over-splitting cost grows monotonically with scale ($+0.026/+0.047/+0.071$ over the $H=4-8$ optimum), mirroring how the single-query compromise grows with width: each head now routes a subspace narrower than what any KV group consumes, so coherent features are split across independently-routed slices. Validation loss is therefore *U-shaped* in H , bottoming out in a flat basin at $H=4-8$ —head count is a real design axis, and $H=4$ or $H=8$ is its near-optimal setting across scales (we adopt $H=8$ at scale; full sweep in §4).

Compute-equivalent gain. The loss deltas of Table 1 can also be expressed as compute-equivalent gain [Davidson et al., 2023, Microsoft AI, 2026]: the factor by which baseline training FLOPs would have to grow to match MHAR’s loss, under a power law $L=AC^{-\alpha}$ fitted to the lower envelope of the three baseline training curves in the style of Kaplan et al. [2020] ($\alpha=0.089$; the cosine schedule makes mid-run points pessimistic, so this envelope is a *conservative* exponent). Because MHAR’s routing adds only 0.5–1.2% FLOPs, the loss gain translates into a $1.3\times/1.8\times/1.7\times$ compute equivalence at 100M/350M/1B. The exponent brackets the estimate: fitting only the three final points gives $\alpha=0.053$ —nearly identical to the 0.057 that Kimi [2025] fit on their own ladder—and would give $1.5-2.6\times$, while a steeper Chinchilla-like $\alpha=0.15$ gives $1.15-1.4\times$.

Beyond the final numbers, MHAR dominates the baseline *throughout* training, not just at convergence: its loss curve stays below the baseline from early in training onward (Figure 11, Appendix H; the 100M web-replication pair shown). Since the two runs share node, software, data order, and global batch, the separation is attributable to the routing mechanism alone.

When do the extra heads matter? A single routing query shares one depth distribution across all d channels—a forced compromise whose cost depends on how much independent subspaces disagree about what to read (probed in Appendix D). The cost is strongly *data-dependent*: on the anneal corpus a zero-initialized single query already captures most of the routing gain (Table 1; MHAR–single-head = $-0.001/-0.028/-0.006$), whereas in the web-corpus replication single-head routing degrades from helpful at 100M to *worse than the plain residual* at 1B while MHAR keeps its full gain (Appendix Table 10). MHAR is thus the best variant in every cell of both distributions: the multi-head reshape costs nothing and acts as insurance against the single-query compromise wherever it does bind. Figure 4 shows the trained MHAR heads use their per-slice freedom: each maintains a stable, distinctive deviation from the head-consensus routing—the H heads carry H different links, not H copies of one.

Downstream evaluation. To check that the loss improvement is not an artifact of the training distribution, we evaluate the anneal-trained baseline and MHAR checkpoints zero-shot (single run) at 100M, 350M, and 1B on held-out WikiText-2 perplexity, LAMBADA [Paperno et al., 2016], and HellaSwag [Zellers et al., 2019] with the LM evaluation harness [Gao et al., 2024] (Table 2). Because the models are trained on the anneal mix and the benchmarks are web-derived, this is a *cross-distribution* transfer test. At *every* scale MHAR improves perplexity and LAMBADA accuracy, with HellaSwag better at 350M and 1B and within sampling noise at 100M. The val-loss gain therefore transfers off the training distribution: the relative perplexity gain roughly doubles from 100M to 350M and holds at 1B (-9% , -20% , -19%), and the LAMBADA gain grows monotonically with scale ($+2.4$, $+3.8$, $+7.2$ points); broader benchmarks are future work.

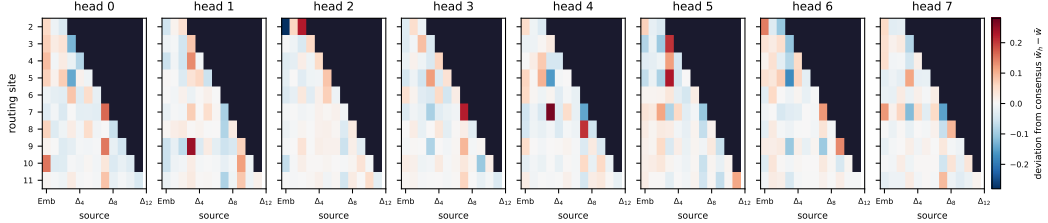


Figure 4: **The H heads carry genuinely different depth-links.** Each head’s token-averaged deviation $\bar{w}_h - \bar{w}$ from the head-consensus routing at the first ten routing sites of the trained 1B MHAR model (dark cells: source not yet available). Deviations reach ± 0.28 vs. 0.067 under a matched-norm random query, replicate on disjoint evaluation text ($r=0.77$), and are near-uncorrelated across heads.

Table 2: Zero-shot downstream evaluation at 100M, 350M, and 1B (single run) of the anneal-corpus models of Table 1. The benchmarks are web-derived and held out from the training mix, so gains here are *cross-distribution* transfer. Each model is evaluated at its training context (100M: seq. 2048; 350M/1B: seq. 1024), so perplexity is comparable only *within* a scale. MHAR improves perplexity and LAMBADA at every scale and HellaSwag at 350M/1B (marginally lower at 100M, within the ± 3 pt noise of 200 samples).

Scale	Method	WikiText-2 PPL ↓	LAMBADA ↑	HellaSwag ↑
100M	Baseline	60.0	9.6%	35.0%
100M	MHAR	54.5	12.0%	33.0%
350M	Baseline	64.9	8.8%	34.5%
350M	MHAR	52.0	12.6%	36.0%
1B	Baseline	56.4	8.8%	30.5%
1B	MHAR	45.4	16.0%	33.0%

3.3 Mid-training

The from-scratch experiments above isolate the routing mechanism at 100M–1B. We now ask whether MHAR survives a realistic 8B *mid-training* (continual-pretraining) regime, in which an existing base model is annealed on a capability-oriented corpus. This also stresses the identity-preserving conversion: we graft multi-head attention residuals onto a pretrained Transformer and cool the learning rate to convergence.

Data. We assemble `anneal_pt_v3`, an English-only anneal corpus of ≈ 1.9 T tokens, from public deduplicated, quality-filtered corpora, deliberately synthetic-, STEM-, and code-heavy in the style of Nemotron anneal blends [Su et al., 2024]. The blend draws on the Nemotron-CC-v2 web crawl and its synthetic rephrasing and diverse-QA variants [NVIDIA, 2025b], the Nemotron pre-training SFT/STEM/code sets [NVIDIA, 2025c], Nemotron-CC-Math [NVIDIA, 2025a], FinePDFs restricted to its top three of twenty quality bins [Hugging Face, 2025a], Stack-Edu code [Hugging Face, 2025b], arXiv, and English Wikipedia, shuffled at the document level into 2,048 uniform shards. Every document retains its source tag, so the composition is measured directly from the shards rather than estimated: Table 13 (Appendix I) reports per-source text-byte shares over a uniform random sample of 48 shards.

Base model and schedule. We start from Marin-8B [Marin Community, 2025] (`marin-8b-base`, revision `phoenix`), a Llama-3.1-8B model (8.03B parameters, $d=4096$, $L=32$, 32/8 grouped-query heads) released at its Warmup–Stable–Decay *pre-decay plateau*—the natural resume point for decay-phase mid-training. The *plain-CPT* control cools from peak 4×10^{-4} to 4×10^{-6} over 9,500 steps at a global batch of ≈ 1.05 M tokens (≈ 10 B tokens total, $\sim 0.5\%$ of the corpus); the MHAR run replicates the control’s schedule exactly—the same peak rate, decay shape, batch, data order, and budget—giving a *schedule-matched* pair that isolates the routing effect. Optimizer and systems details (cold-start AdamW with re-warmup, FSDP, EMA, z-loss) are in Appendix I.

Identity-preserving conversion. We convert with the *delta* (additive-stream) form of attention residuals [Luo et al., 2026], which *adds* the routed term to the intact residual stream (h =partial + $\alpha \cdot$ routed) rather than *replacing* it as the from-scratch block/full form does (Appendix E); closing the gate therefore leaves the pretrained computation untouched. The routed sources are *block-aggregated*: a learnable null source plus one delta per block of four consecutive layers (`num_blocks`=8 for the 32-layer model), so each routing site attends over at most nine sources with H =8 heads. MHAR is grafted onto the pretrained model with a zero-initialized output gate (α =0 at init), so the converted model is *numerically identical* to the base at step 0 (maximum logit deviation 0 in fp32), and the routing opens during mid-training. This lets us continue-train an existing checkpoint without a loss spike: on the schedule-matched pair (identical seed and data order) both runs start from the same loss (1.8616 vs 1.8615 at the first logged step), and the per-step gap stays $36\times$ below batch noise across all 9,500 steps.

Downstream protocol. We evaluate six capability-oriented tasks with the LM Evaluation Harness [Gao et al., 2024], grouped as *General* —MMLU [Hendrycks et al., 2021a] (57-subject average) and GPQA [Rein et al., 2024] (main split, 0-shot)—and *Math & Coding*: GSM8K [Cobbe et al., 2021] (5-shot, strict exact match), MATH [Hendrycks et al., 2021b] (Minerva 4-shot [Lewkowycz et al., 2022]), and HumanEval [Chen et al., 2021] and MBPP [Austin et al., 2021] (pass@1, greedy). MATH is scored with the format-robust `math_verify` checker rather than the harness’s template-strict extractor: annealed models increasingly answer in `\boxed{}` style while abandoning the few-shot “Final Answer” template, which strict extraction mistakes for failure despite mathematically correct solutions. For generative tasks we report paired per-item exact McNemar tests between the schedule-matched arms. Table 3 reports final (EMA) checkpoints.

Table 3: Downstream accuracy ($\uparrow$) after 8B mid-training on `anneal_pt_v3` (final checkpoints, EMA weights). The control and MHAR columns are *schedule-matched*: identical LR schedule, batch, data order, and ≈ 10 B-token budget.

	Task	Base	Plain CPT	+MHAR
<i>General</i>	MMLU	0.554	0.643	0.645
	GPQA	0.266	0.315	0.346
<i>Math & Coding</i>	GSM8K	0.190	0.470	0.502
	MATH	0.053	0.191	0.191
	HumanEval	0.122	0.409	0.415
	MBPP	0.148	0.386	0.392

Mid-training on the anneal corpus produces large downstream gains (GSM8K more than doubles, $19.0 \rightarrow 47.0$; MATH more than triples, $5.3 \rightarrow 19.1$; MMLU rises $+8.9$) while the schedule-matched comparison isolates what routing itself adds: a real gain of $+3.2$ GSM8K (123 vs. 81 discordant items, paired exact McNemar $p=0.004$) and $+3.1$ GPQA (27 vs. 13, $p=0.038$), with MMLU, MATH, and code statistically unchanged. The routing gain at this scale is thus modest and concentrated on GSM8K and GPQA; how it interacts with a more aggressive re-training schedule is left to future work. We release the mid-trained MHAR model and its schedule-matched control at <https://huggingface.co/wdlctc/marin-8b-cpt-mhar-delta> and <https://huggingface.co/wdlctc/marin-8b-cpt-plain-lr4e4>.

3.4 Parameter and compute cost

Parameter cost. MHAR adds only $+0.02\%$ parameters over the vanilla baseline (per-layer routing queries and RMSNorms: $+100\text{K}$ at 350M, $+187\text{K}$ at 1B; Table 4)—far too few to account for the -0.06 to -0.15 nat gain (Table 1). Crucially, the multi-head split itself is *exactly* free over single-head routing in parameters, compute, *and* memory: H merely reshapes one query into H smaller ones, replacing one softmax with H softmaxes over the short depth axis (length $\leq 2L+1$), so single-head and MHAR carry identical throughput and peak memory by construction (Appendix F). Every MHAR-vs-single-head comparison in this paper is therefore not merely an iso-parameter, iso-compute control but iso-*wall-clock*: giving single-head routing more time cannot close the gap, and the improvement cannot come from added parameters, normalization, or compute.

Table 4: **Training speed and memory.** Baseline Transformer vs. MHAR (torch.compile reference kernels) vs. MHAR with our fused routing kernels, at the three model settings of Table 1. Throughput is relative to the same-scale baseline; MHAR is parameter-matched (+0.02%); single-head attention residuals are cost-identical to MHAR by construction. Protocol, absolute medians, and the full breakdown in Appendix F (Table 8).

Method	Throughput (vs. base)			Peak mem. (GB)		
	100M	350M	1B	100M	350M	1B
Baseline Transformer	1.00×	1.00×	1.00×	41.5	19.4	19.0
MHAR	0.54×	0.32×	0.23×	52.4	40.1	47.5
MHAR + fused kernels (ours)	0.88×	0.71×	0.55×	42.0	20.0	20.1

Table 5: **Routing-operation speedup**, isolating the kernels from Table 4’s end-to-end numbers: wall-clock of all routing calls of one microbatch, forward+backward, bf16, single H100; the 8B column is the mid-training delta variant. End-to-end gains are smaller because routing is only part of a training step (batch settings and the eager-reference comparison in Appendix F).

Routing per microbatch (ms)	100M	350M	1B	8B (delta)
torch.compile	30.2	53.1	153.2	651
Fused Triton (ours)	5.7	11.4	42.4	323
Speedup	5.3×	4.7×	3.6×	2.0×

Equal-compute comparison. The one comparison that is *not* iso-compute is the attention-residual family vs. the plain baseline: materializing and routing over the list of past sublayer outputs has a mechanism cost, inherited from Kimi [2025] rather than introduced by the multi-head split, so we report the improvement *over the plain Transformer* at equal steps. Our fused routing kernels (Appendix E) remove most of this cost (Table 4; the routing operation in isolation speeds up by 2.0–5.3× over torch.compile, Table 5): MHAR trains at 0.88×/0.71×/0.55× baseline throughput at 100M/350M/1B with approximately baseline peak memory, so an equal-wall-clock baseline corresponds to ~1.1–1.8× more steps; a direct iso-compute study against the plain baseline is future work.

4 Ablations

The head count has a two-sided optimum. Figure 5a sweeps the routing-head count at 1B (KV = 8) with everything else fixed: loss falls sharply from $H=1$ (3.270) into a flat basin at the KV granularity, reaching its optimum at $H=4$ (3.132) and $H=8$ (3.129)—the two are essentially tied, within tail-mean noise—then rises past it: $H=16$ (3.173) sits 0.044 above the optimum, beyond the tail-mean noise ($\sim \pm 0.02$), and the same overshoot appears in every KV row of the grid (+0.031–0.052). The two failure modes are granularity mismatches with the stream’s consumers, though asymmetric ones: too few heads force unrelated subspaces through one depth distribution (costing 0.14 at $H=1$), while too many split what a KV group consumes coherently (costing 0.03–0.05 at $H=16$, still well below single-head). The size of the overshoot is corpus-dependent: on the anneal corpus of Table 1 it is larger and $H=16$ does land above single-head routing. The optimum lands at $H=4$ or $H=8$ at every scale we tested—a flat basin—and for large-scale models we adopt $H=8$.

Routing should be multi-head, just like attention. The attention-residual bypass is structurally a self-attention query over *depth* (Eq. 2) and inherits the same limitation that made attention multi-head: one query cannot serve every subspace at once (§1). The thesis of our work follows: **if attention is multi-head, the attention-residual bypass should be multi-head too.** This section shows that splitting the routing query into H heads is not merely analogous but the only setting that stays robust as models grow and the data distribution changes (§3), and that the optimum consistently sits at $H=4$ or $H=8$ (a flat basin, from which we adopt $H=8$ at scale), capturing essentially all of the benefit.

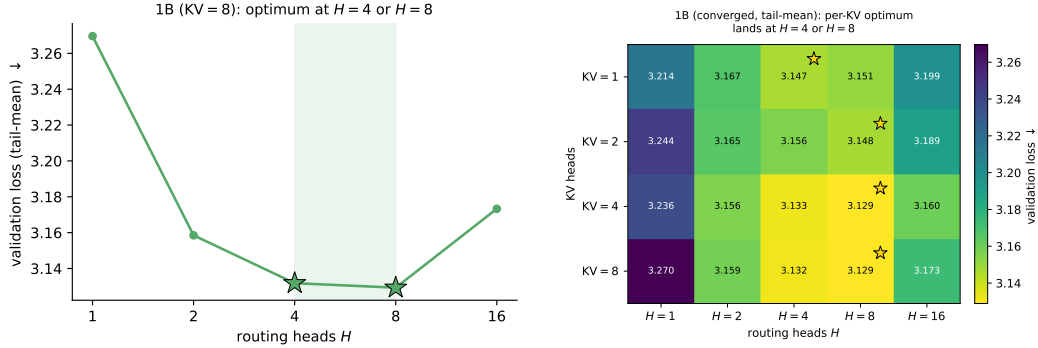


Figure 5: **Head count at 1B: a flat $H=4-8$ optimum.** (a) The $KV=8$ row (tail-mean): loss falls from single-head ($H=1$) into a flat basin at $H=4-8$ —the optimum, 3.132 and 3.129, essentially tied—then rises past it: $H=16$ (3.173) gives back roughly a third of the multi-head gain, though it stays well below $H=1$. (b) The full $kv \times H$ grid: multi-head routing helps at every KV, the per-KV optimum (gold stars) lands at $H=4$ or $H=8$, and the $H=16$ column overshoots the basin in every row while staying below $H=1$.

How many heads: a convergence signature. How many routing heads help, and where the optimum sits, depends on how converged the router is. In a deliberately *under-trained* grid (100M on the web corpus at its tuned LR 1×10^{-3} but only 5K steps, vs. the 20K used elsewhere; Figure 8), the clean signal is in the most over-subscribed row, $KV=1$: there the single shared query ($H=1$) is worst and $H=8$ best, a *monotone* 0.028 drop, well above the ~ 0.006 single-seed noise floor (the higher-KV rows are within noise). Before convergence, more heads only ever help. Trained to convergence, the optimum instead *saturates* into a flat $H=4-8$ basin: the full grids at 100M (Appendix C) and 1B (Figure 5b) put every per-KV optimum at $H=4$ or $H=8$ (the over-subscribed $KV \in \{1, 2\}$ rows still prefer more heads even at convergence). We therefore adopt $H=8$ for large-scale models (Table 1). This is exactly what the forced-compromise account predicts: the penalty that makes splitting past KV unhelpful is *learned* (Appendix D) and only binds once feature subspaces have specialized—before convergence extra heads are free averaging flexibility; after it, $H=4-8$ is the knee. Practically, $H=8$ is a safe default for a *well-tuned* run; under-tuning only ever pushes the optimum to *more* heads, never fewer, so the choice never costs you.

Single-head routing is not robust across data distributions. Single-head routing ($H=1$) is exactly the original attention-residual formulation [Kimi, 2025]: one learned query routes the depth history. On the anneal corpus it is strong (Table 1), but in the web-corpus replication—read against the baseline with *each method at its own optimal learning rate* (Appendix Table 11)—it is *not* robust as the model grows: it helps at 100M (-0.039) but lands above the baseline at both larger scales ($+0.038$ at 350M, $+0.105$ at 1B). MHAR, in contrast, improves at every scale on *both* corpora; on web data multi-head’s advantage over single-head routing *widens* monotonically with scale: -0.010 , -0.118 , -0.168 at 100M/350M/1B (best-vs-best; larger still at a shared rate). The zero-parameter reshape is what makes attention residuals robust—the central claim of this work.

5 Conclusion

Multi-head attention residuals remove the forced-compromise bottleneck of a single routing query by giving each feature subspace its own depth-routing head, at zero parameter cost over single-head routing. Trained from scratch from 100M to 1B on a quality-filtered anneal corpus, MHAR improves over a standard Transformer at every scale with gains that grow from 100M to the larger scales ($-0.061/-0.149/-0.140$), and it is the only variant whose gain also survives web-generic pretraining data, where a single routing query degrades from helpful to harmful and the multi-head advantage widens with scale; the gain transfers to held-out perplexity and LAMBADA accuracy. The optimum consistently lands at $H=4$ or $H=8$ across scales, and for large-scale models we adopt $H=8$ (aligning routing to attention head groups adds nothing measurable; Appendix D). Beyond from-scratch training, an identity-preserving conversion carries MHAR to 8B mid-training (GSM8K

+3.2, paired $p=0.004$), and fused routing kernels cut the mechanism’s systems cost to $0.55\text{--}0.88\times$ baseline throughput at near-baseline peak memory—together making multi-head depth routing practical beyond research scale. Understanding *why* the optimum sits at $H=4\text{--}8$, and what individual routing heads learn to attend to, are natural next steps.

References

- Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *EMNLP*, 2023.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Thomas Bachlechner, Bodhisattwa Prasad Majumder, Huanru Henry Mao, Garrison W Cottrell, and Julian McAuley. Rezero is all you need: Fast convergence at large depth. *UAI*, 2021.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.
- Tom Davidson, Jean-Stan Denain, Pablo Villalobos, and Guillem Bas. AI capabilities can be significantly improved without expensive retraining. *arXiv preprint arXiv:2312.07413*, 2023.
- DeepSeek. Manifold-constrained hyper-connections. *arXiv preprint arXiv:2512.24880*, 2025.
- Nelson Elhage, Neel Nanda, Catherine Olsson, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *JMLR*, 2022.
- Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muenighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, 2024. URL <https://zenodo.org/records/10256836>.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016.
- Ruining He, Anirudh Ravula, Karol Kanber, and Joshua Ainslie. Realformer: Transformer likes residual attention. *ACL Findings*, 2021.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *ICLR*, 2021a.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks Track*, 2021b.
- Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *CVPR*, 2017.

Hugging Face. Finepdfs. <https://huggingface.co/datasets/HuggingFaceFW/finepdfs>, 2025a.

Hugging Face. Stack-edu. <https://huggingface.co/datasets/HuggingFaceTB/stack-edu>, 2025b.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

Kimi. Attention residuals. *arXiv preprint arXiv:2603.15031*, 2025.

Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, et al. Solving quantitative reasoning problems with language models. In *Advances in Neural Information Processing Systems*, 2022.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *ICLR*, 2019.

Cheng Luo, Zefan Cai, and Junjie Hu. Delta attention residuals. *arXiv preprint arXiv:2605.18855*, 2026.

Marin Community. Marin: An open lab for building foundation models. <https://huggingface.co/marin-community/marin-8b-base>, 2025.

Microsoft AI. MAI-Thinking-1: Building a hill-climbing machine. Technical report, Microsoft, 2026. URL <https://microsoft.ai/pdf/mai-thinking-1.pdf>.

NVIDIA. Nemotron-cc-math-v1. <https://huggingface.co/datasets/nvidia/Nemotron-CC-Math-v1>, 2025a.

NVIDIA. Nemotron-cc-v2. <https://huggingface.co/datasets/nvidia/Nemotron-CC-v2>, 2025b.

NVIDIA. Nemotron-pretraining-sft-v1. <https://huggingface.co/datasets/nvidia/Nemotron-Pretraining-SFT-v1>, 2025c.

Matteo Pagliardini, Pierre Ablin, and Martin Jaggi. Denseformer: Enhancing information flow in transformers via depth weighted average. *arXiv preprint arXiv:2402.02622*, 2024.

Denis Paperno, Germán Kruszewski, Angeliki Dutta, Raffaella Bernardi, Gemma Boleda, Raquel Fernández, and Marco Baroni. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *ACL*, 2016.

Guilherme Penedo, Hynek Kydlíček, et al. The fineweb datasets: Decanting the web for the finest text data at scale. *arXiv preprint arXiv:2406.17557*, 2024.

Zihan Qiu, Zekun Wang, Bo Zheng, Zeyu Huang, Kaiyue Wen, Songlin Yang, Rui Men, Le Yu, Fei Huang, Dayiheng Liu, Jingren Zhou, and Junyang Lin. Gated attention for large language models: Non-linearity, sparsity, and attention-sink-free. *arXiv preprint arXiv:2505.06708*, 2025.

Qwen. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*, 2024.

Anton Razzhigaev, Matvey Mikhalechuk, Elizaveta Goncharova, Ivan Gerasimov, Ivan Oseledets, Denis Kuznetsov, and Alexander Korotin. Your transformer is secretly linear. In *ACL*, 2024.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof Q&A benchmark. In *Conference on Language Modeling (COLM)*, 2024.

- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maarten, Quoc V Le, Jeff Dean, and Geoffrey E Hinton. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *ICLR*, 2017.
- Rupesh K Srivastava, Klaus Greff, and Jürgen Schmidhuber. Training very deep networks. *NeurIPS*, 2015.
- Dan Su, Kezhi Kong, Ying Lin, Joseph Jennings, Brandon Norick, Markus Kliegl, Mostofa Patwary, Mohammad Shoeybi, and Bryan Catanzaro. Nemotron-CC: Transforming Common Crawl into a refined long-horizon pretraining dataset. *arXiv preprint arXiv:2412.02595*, 2024.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017.
- Andreas Veit, Michael J Wilber, and Serge Belongie. Residual networks behave like ensembles of relatively shallow networks. *NeurIPS*, 2016.
- Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Dongdong Zhang, and Furu Wei. Deepnet: Scaling transformers to 1,000 layers. *arXiv preprint arXiv:2203.00555*, 2022.
- Da Xiao, Qingye Meng, Shengping Li, and Xingyuan Yuan. Muddformer: Breaking residual bottlenecks in transformers via multiway dynamic dense connections. *ICML*, 2025.
- Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. *ICML*, 2020.
- Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *ACL*, 2019.
- Yifan Zhang, Yifeng Liu, Mengdi Wang, and Quanquan Gu. Deep delta learning. *arXiv preprint arXiv:2601.00417*, 2026a.
- Yifan Zhang, Yifeng Liu, Mengdi Wang, and Quanquan Gu. Residual stream duality in modern transformer architectures. *arXiv preprint arXiv:2603.16039*, 2026b.
- Defa Zhu et al. Hyper-connections. *arXiv preprint arXiv:2409.19606*, 2024.

A Related Work

Residual and dense connections. Residual connections [He et al., 2016] and their gated predecessor, highway networks [Srivastava et al., 2015], enable very deep training by providing additive identity paths; Veit et al. [2016] characterize residual networks as ensembles of paths of varying length. DenseNets [Huang et al., 2017] instead concatenate all previous feature maps, giving each layer direct access to every earlier one. Our sources list $\mathcal{S}$ is dense in this spirit, but rather than concatenating (which grows width) or summing (which is the standard residual stream), we *route* over the sources with a learned softmax, keeping width fixed while making each past output individually addressable.

Cross-layer aggregation in Transformers. Several works move beyond the single additive stream. DenseFormer [Pagliardini et al., 2024] adds a learned depth-weighted average of all previous block outputs; MUDDFormer [Xiao et al., 2025] learns multi-way, input-dependent dense connections between layers; RealFormer [He et al., 2021] carries a residual path on the attention scores. ReZero [Bachlechner et al., 2021] and DeepNet [Wang et al., 2022] instead rescale the standard residual branch for stable deep training. Attention residuals differ by replacing the additive stream with an explicit softmax *retrieval* over past sublayer outputs: unlike DenseFormer’s fixed depth-weighted average, the retrieval is input-adaptive and—crucially for us—*splittable* into per-subspace heads. Making that retrieval multi-headed at no parameter or FLOP cost is our contribution.

Routing over layer outputs. Closest to our setting are attention residuals [Kimi, 2025], which introduce a learned attention over the history of layer outputs. We build directly on this formulation and identify its single routing query as a bottleneck: splitting it into independent per-subspace heads (Eq. 3) improves quality while staying parameter- and FLOP-matched. Hyper-connections [Zhu et al., 2024] generalize residual connections into a learnable width- n set of parallel streams, and manifold-constrained hyper-connections [DeepSeek, 2025] add geometric constraints to their mixing; these enrich the *connection topology* between layers, whereas we keep a single stream and enrich the *routing* that reads the depth history—our heads act on routing distributions, not on separate streams. MHAR outperforms hyper-connections at matched recipe across all three scales (Table 1). Concurrent work attacks the *same* bottleneck on a different axis: routing over per-sublayer *deltas* rather than cumulative states de-correlates the layer sources [Zhang et al., 2026a]—targeting the *secondary, hypothesized* source-collinearity factor of our forced-compromise cost—and Zhang et al. [2026b] analyze residual-stream structure. We instead split the routing query across feature subspaces, targeting the *primary* width-disagreement factor that our probe directly supports; the two fixes are complementary.

Conditional computation and mixtures. Mixture-of-experts routing [Shazeer et al., 2017, Fedus et al., 2022] and mixture-of-depths [Raposo et al., 2024] also use learned routing, but route *tokens* to experts or layers to save or allocate compute. Our routing is over the *depth axis* of already-computed sublayer outputs and is applied to every token; it changes what each sublayer reads, not which tokens are processed.

Multi-head mechanisms and interpretability. Multi-head attention [Vaswani et al., 2017] lets different heads attend to different token-level patterns; we apply the same intuition one level up, letting different heads attend to different *layers*. Mechanistic analyses of the residual stream as a shared communication channel that many components read from and write to [Elhage et al., 2021]—and evidence that transformer layers act near-linearly on the stream [Razzhigaev et al., 2024]—motivate giving different subspaces independent read access to depth, which is precisely what multi-head routing provides. Methodologically, validating a lightweight, parameter-cheap architectural change through systematic from-scratch experiments across scale follows recent studies of attention modifications such as gated attention [Qiu et al., 2025].

B Attention residuals as the single-head route

MHAR changes *only* the routing kernel; the forward pass (Figure 3) is exactly that of attention residuals [Kimi, 2025]. Figure 6 gives the single-head kernel: one shared query produces one softmax over depth that all d coordinates read through. Dropping it in for route in the *identical* forward yields attention residuals exactly; equivalently, it is the multi-head route of Figure 3 at `self.H=1` (a single head of width d). This is the *single-head* method reported in every table, so the MHAR-vs-single-head comparison is a strict iso-parameter, iso-forward control that isolates the multi-head split.

```

1 def route(self, sources: list[Tensor], proj: Linear, norm: RMSNorm):
2     """Single-head depth routing = attention residuals (Kimi 2025):
3         one shared query, one softmax over depth, read by all D coords.
4         Equals the MHAR route at H=1."""
5     V = torch.stack(sources) # [N, B, T, D]
6     logits = einsum('d, n b t d -> n b t', proj.weight.view(-1), norm(V))
7     return einsum('n b t, n b t d -> b t d', logits.softmax(0), V)

```

Figure 6: **Single-head route (attention residuals).** A drop-in replacement for the multi-head route of Figure 3 that leaves forward unchanged: a single query yields one depth distribution shared by all d coordinates. This is exactly MHAR with $H=1$, and the single-head baseline in all our tables.

C Routing-head sweep at 100M

Figure 7 shows the full 4×4 routing-head sweep at 100M ($H, KV \in \{1, 2, 4, 8\}$), the converged counterpart of the under-trained grid in Figure 8. These grids train on the web replication corpus (FineWeb-Edu, randomly initialized routing queries; Appendix H), so their absolute losses are not comparable to the anneal-corpus Table 1. At 100M the $KV=4$ and $KV=8$ optima sit at $H=4$ and $H=8$ (gold stars; on the red $H=KV$ line), while at small $KV \in \{1, 2\}$ more heads help, up to $H=8$. This grid pins down the two practically relevant conclusions: multi-head routing helps at every KV setting, and the optimum consistently lands at $H=4$ or $H=8$ (we adopt $H=8$ at scale). We ran the same sweep at 350M and 1B. At 350M the $KV \geq 4$ optima again fall at $H=8$. At 1B, however, the global grid argmin is the *off*-diagonal cell $KV=2, H=8$ (tail-mean 3.258), and the four lowest cells—the $KV2/H8$, $KV8/H8$, $KV8/H4$ and $KV4/H8$ —all at $H=4$ or $H=8$ —lie within ~ 0.02 of one another, below the $\sim \pm 0.07$ single-pass eval noise; these sweeps are also single-seed. We therefore read $H=8$ at 1B as *within noise of optimal* rather than the strict grid minimum, and summarize the two sweeps here rather than as separate figures.

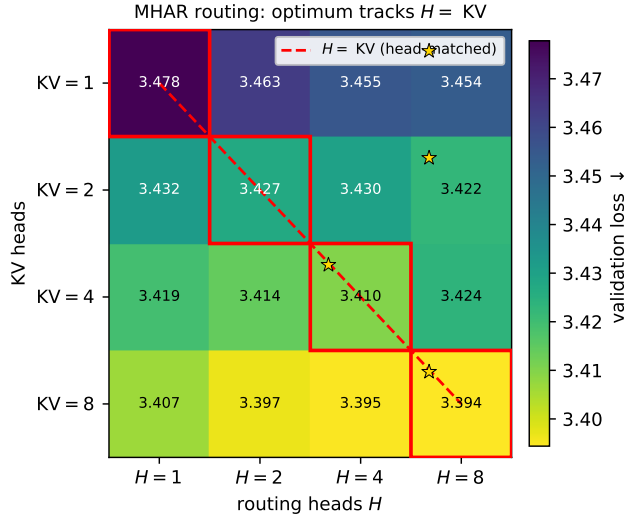


Figure 7: Validation loss of MHAR at 100M as a function of routing heads H (x) and KV heads (y). Lower is better. Gold stars mark the best H per KV; the red dashed line is the head-matched diagonal $H=KV$. At 100M the $KV=4$ and $KV=8$ optima sit at $H=4$ and $H=8$ (we adopt $H=8$ at scale). The 100M architecture of Table 1, trained on FineWeb-Edu at 5×10^{-4} .

D Why single-head routing breaks: the forced-compromise mechanism

The pattern above has a single explanation. In the web-data setting (Table 10, whose checkpoints this appendix probes), multi-head routing buys almost nothing over single-head at 100M—the H -axis is flat, MHAR beating single-head by only 0.010 (Appendix C), while both already beat the baseline, so one head suffices—yet is worth ~ 0.17 by 1B, where the single query has crossed from helpful (-0.039) to harmful ($+0.105$, best-vs-best). A single $(d,)$ query routes the entire width through one softmax α (Eq. 2), serving every subspace with one depth distribution. We call the resulting penalty a *forced compromise*. Its primary, empirically-supported driver is (i) how differently independent subspaces would route if freed—which grows with model *width* (more KV subspaces) and which we isolate directly below. A second, hypothesized factor is (ii) how *collinear* the candidate sources are (near-collinear sources sharpen the shared softmax and make any disagreement costlier to average away); this would in principle grow with the cumulative source count ($N=2L+1$), but we do not cleanly isolate it, and our probe finds it *non-monotonic* across scale (Table 6). Factor (i) alone already predicts the observed crossover—and predicts that splitting the query into H heads, each paying the compromise only inside its own d/H slice, removes the harm. That is exactly what MHAR does.

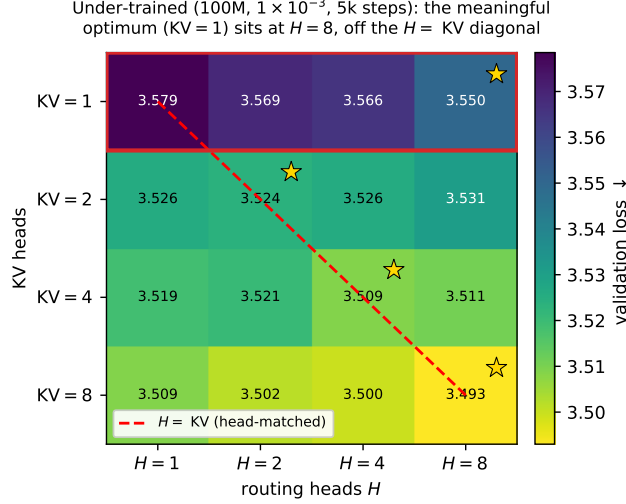


Figure 8: **Under-trained 100M $kv \times H$ grid** (web replication, 1×10^{-3} , only 5K steps; tail-mean 3500–5000)—the under-trained counterpart of Figure 7. Before the router converges, more heads only ever help: the clean supra-noise signal is the most over-subscribed row KV = 1 (red outline; a monotone 0.028 drop from $H=1$ to $H=8$, above the ~ 0.006 single-seed noise floor), while the higher-KV rows are within noise. Gold stars: per-KV optimum; red dashed line: $H = KV$.

Direct confirmation on the trained queries. We measure both factors directly on the trained single-head ($H=1$) checkpoints, with *no* new training (Table 6). At each routing site we split the trained query q into S contiguous slices and compute, for each slice, the depth distribution it would prefer alone, $a_s = \text{softmax}_n(q_{[s]}, \text{RMSN}(\mathbf{s}_n)_{[s]})$, relative to the shared α the single softmax actually uses. The mean $\text{KL}(a_s \parallel \alpha)$ is the latent *width-disagreement*. Three controls make the reading clean.

(i) *The disagreement is learned, not geometric.* A matched-norm *random* query produces only ~ 0.03 – 0.10 of the disagreement, versus 0.27 – 0.70 for the trained query, so the signal is overwhelmingly learned rather than an artifact of source geometry. The learned excess (trained minus null) grows *monotonically* across all four scales ($0.235 \rightarrow 0.281 \rightarrow 0.512 \rightarrow 0.606$ from 100M to 1B) and is largest at 1B—exactly the scale where single-head routing *regresses below baseline*, so the probe measures the mechanism directly at the regression scale. (ii) *Width is the driver.* Widening at *fixed* depth and KV ($d512 \rightarrow d768$, L12, kv4) raises the trained-query KL by 14% ($0.273 \rightarrow 0.311$) while the null stays flat, so the learned excess rises 20% ($0.235 \rightarrow 0.281$)—and here d alone changes. (iii) *Not a “more sources” artifact.* A matched-source-count control leaves a $\sim 2\times$ median KL growth, and contiguous vs. head-interleaved slicing give identical results (echoing the head-alignment null of Table 7). Source collinearity is *non-monotonic* across scale—it rises from 100M to 350M ($0.060 \rightarrow 0.114$) but then *drops* at 1B (0.087), partly a dimension effect (cosine shrinks at larger d)—so it is *not* a clean cross-scale driver, and the learned width-disagreement is the signal we rely on. The forced-compromise cost thus grows with scale and peaks at 1B, tracking the learned disagreement rather than collinearity, consistent with the loss crossover in Table 10.

Crucially, **the disagreement is not merely latent**: it moves the loss. At fixed depth (L12, kv4), the same widening $d512 \rightarrow d768$ that raises the probed width-disagreement ($0.235 \rightarrow 0.281$) also widens MHAR’s validation-loss advantage over single-head routing from -0.009 to -0.020 , while single-head’s edge over the baseline erodes ($-0.041 \rightarrow -0.033$)—a matched-recipe training control that isolates width in loss exactly as the probe isolates it in disagreement (single-seed; Appendix G). *Scope*: the slices are sub-parts of *one* trained query rather than independently trained routers, so the KL is an upper-bound proxy for the disagreement of fully independent heads.

The probe measures the disagreement a single query must *suppress*; Figure 4 (main text) shows the complement on the trained 1B MHAR model: the freed heads maintain stable, mutually distinct deviations from the head-consensus routing—they route differently, not redundantly.

Table 6: Direct probe of the trained *single-head* routing queries (no new training; $S=4$ slices). *Width-disagreement* is the mean $\text{KL}(a_s \parallel \alpha)$ between each query slice’s preferred depth distribution and the shared α ; a matched-norm *random* query gives the null, and the learned excess (trained–null) is the genuine learned signal. The middle column is a width-isolating control ($d512 \rightarrow d768$ at fixed $L12/\text{kv}4/N$): the learned disagreement rises while the null and collinearity stay flat, so width itself drives it. Across scale the learned excess grows monotonically $2.6\times$ ($0.235 \rightarrow 0.606$) and is the *primary, empirically-supported* factor; source collinearity (mean pairwise cosine of the $N=2L+1$ sources)—a *secondary, hypothesized* factor we do not cleanly isolate—is non-monotonic ($1.9\times$ from 100M to 350M, then *lower* at 1B, partly a dimension effect), so the disagreement factor is the clean cross-scale signal. *Caveat*: slices are sub-parts of one trained query (a proxy for independent heads) rather than independently trained routers.

Probe quantity ($S=4$)	100M	124M (<i>width ctrl</i>)	350M	1B
	$d512/L12/\text{kv}4$	$d768/L12/\text{kv}4$	$d1024/L24/\text{kv}8$	$d1280/L36/\text{kv}8$
Width-disagreement KL (trained)	0.273	0.311	0.570	0.701
random-query null	0.038	0.030	0.058	0.095
learned excess (trained–null)	0.235	0.281	0.512	0.606
Source collinearity (mean cos)	0.060	0.063	0.114	0.087
N sources ($= 2L+1$)	25	25	49	73

Aligning routing to attention head groups. A natural hypothesis is that routing should be *aligned* to the model’s attention structure rather than to arbitrary coordinate slices. We test this with *MHAR-HW*, a head-*wise* variant that gives each grouped-query-attention KV head group its own full-width routing query and feeds that group’s routed mixture to its $Q/K/V$ projections; with one full-width query per group, its routing cost is $G\times$ that of MHAR (G the number of KV groups) but still negligible next to the sublayers. Table 7 compares the two across scales: head-alignment yields *no detectable* improvement over arbitrary subspace routing—the two are within the ± 0.07 eval-noise floor at all three scales (the $+0.016$ at 1B is a single final-step eval). Because this ablation is single-seed and within noise, it does not resolve a difference in either direction: we read it as a *failure to find* a benefit from head-alignment, not as evidence of equivalence. It is nonetheless consistent with the benefit of multi-head routing coming from routing the depth history through multiple independent heads rather than from tying those heads to the attention head groups.

Table 7: Head-alignment ablation (FineWeb-Edu; validation loss $\downarrow$, final-step eval): routing over arbitrary subspaces (MHAR) vs. aligned to KV head groups (MHAR-HW), with matched head count. Aligning to attention heads shows no measurable benefit over arbitrary subspace routing; all differences fall within the ± 0.07 eval-noise floor (single-seed, final-step). Separate matched-run batch (§3); read the within-table Δ .

Scale	MHAR	MHAR-HW	$\Delta(\text{hw} - \text{mh})$
100M	3.330	3.333	+0.003
350M	3.225	3.223	−0.002
1B	3.213	3.229	+0.016

E Fused routing kernels

Depth routing is memory-bound for two reasons. First, it does almost no arithmetic per byte: scoring a source row against the query and folding it into the mix (Eq. 3) is a constant number of operations per element, with none of the data reuse that lets the attention and MLP matrix multiplies run compute-bound, so throughput is set by memory bandwidth. Second, the traffic is inherently quadratic in depth: sublayer s scores and mixes all $N_s=s+1$ prior states, so a training step must move $\sum_s N_s B T d = O(L^2 B T d)$ of source data no matter how it is implemented. A reference implementation pays this bound several times over (per-sublayer stacking of the source list, a materialized normalized key tensor, separate softmax/mix kernels, thousands of gradient-accumulation launches in backward) and additionally retains the per-sublayer stacked sources for autograd, the dominant activation cost of the mechanism (Appendix F). We provide a fused Triton implementation that reaches (approximately) the bound once per pass: a shared source buffer plus one kernel

per routing call in each direction, with deterministic reductions throughout (no atomics), so runs are bitwise reproducible.

```

1 def route_fwd(V, q, g, partial, out, W): # one program per position p
2     m = full([H], -inf); l = zeros([H]); acc = zeros([H, K])
3     for n in range(N): # single pass over the depth sources
4         v = load(V[n, p]) # [H, K] tile; the only source read
5         r = rsqrt(mean(v * v) + eps) # RMS statistic over the full d-row
6         khat = round(v * r) * g # fused RMSNorm; no key tensor made
7         logit = sum_k(q * khat) # per-head scores [H]
8         W[n, p] = logit # raw logit; normalized after loop
9         m_new = maximum(m, logit) # online softmax: rescale, fold in
10        acc = acc * exp(m - m_new) + exp(logit - m_new) * v
11        l = l * exp(m - m_new) + exp(logit - m_new)
12        m = m_new
13    out[p] = acc / l + (partial[p] if DELTA else 0)
14    for n in range(N): # tiny second loop: N*H scalars
15        W[n, p] = exp(W[n, p] - m) / l # exact weights, saved for backward

```

Figure 9: **Fused routing forward.** Online softmax over the depth sources with the RMSNorm fused in: each source row is read once, nothing is materialized except the $[N, B, T, H]$ routing weights W (a factor d/H smaller than the stacked sources a reference path retains for autograd).

```

1 def route_bwd(V, W, dout, dV, dq_part, dg_part): # one program per pos. p
2     do = load(dout[p]) # [H, K]
3     S = zeros([H])
4     for n in range(N): # pass 1: softmax coupling term
5         v = load(V[n, p]); w = W[n, p]
6         S += w * sum_k(do * v)
7     dq = zeros([H, K]); dg = zeros([H, K])
8     for n in range(N): # pass 2: all gradients
9         v = load(V[n, p]); w = W[n, p]
10        dlogit = w * (sum_k(do * v) - S) # softmax backward [H]
11        r = rsqrt(mean(v * v) + eps) # norm recomputed, not saved
12        khat = round(v * r)
13        dq += dlogit * (khat * g) # query grad, register-resident
14        dK = dlogit * q # grad wrt normalized key
15        dg += dK * khat # norm-weight grad
16        dkp = dK * g # grad wrt v * r
17        dv = r * dkp - (r**3 / d) * sum(dkp * v) * v + w * do
18        dV[n, p] = dv if FIRST_CALL else dV[n, p] + dv # in-place accum.
19    dq_part[p] = dq; dg_part[p] = dg # per-position partials, host-summed

```

Figure 10: **Fused routing backward.** Two passes over the source rows: the first accumulates the softmax coupling term S , the second recomputes the norm from the buffer (the row is re-read anyway) and emits the closed-form key- and value-path source gradients, accumulating them across routing calls in place (safe by autograd’s reverse-order execution; asserted at runtime).

Shared source buffer. Each forward pass allocates one $[2L+1, B, T, d]$ buffer and writes every residual state into it exactly once as it is produced. Routing call s reads rows $[0, N_s)$ directly, eliminating the per-sublayer stack (an $O(N_s)$ copy per call, $O(L^2 B T d)$ in total).

Forward: online softmax over depth with the norm fused in. One program per position (b, t) holds a $[H, d/H]$ accumulator in registers and loops over the N source rows. For each row it computes the RMS statistic and the per-head logits $\ell_{n,h} = \langle q_h, \widehat{s}_{n,h} \rangle$ on the fly (fp32 arithmetic, with the normalized key rounded to the storage dtype exactly where the reference rounds it), and folds the row into the accumulator with the standard running-max/renormalization update, as in online-softmax attention kernels [Dao et al., 2022]. The normalized keys are never materialized, and the only activation saved for backward is the routing-weight tensor $w \in \mathbb{R}^{N \times B \times T \times H}$ —a factor

d/H smaller than the stacked sources a reference path retains. In delta modes the kernel adds the residual stream to the mix before writing the output.

Backward: one kernel, norms recomputed, gradients accumulated in place. Given $\partial\mathcal{L}/\partial\text{out}$ and the saved w , the kernel makes two passes over the source rows. The first accumulates the softmax coupling term $S_h = \sum_n w_{n,h} \langle \partial\text{out}_h, s_{n,h} \rangle$; the second recomputes the RMS statistic from the buffer (the row is being re-read anyway, so recomputation is free relative to saving keys), forms $\partial\ell_{n,h} = w_{n,h} (\langle \partial\text{out}_h, s_{n,h} \rangle - S_h)$, and emits per-source gradients combining the value path and the key path through the norm: the contribution added to ∂s_n is

$$r (g \odot \partial\hat{k}_n) - \frac{r^3}{d} \langle g \odot \partial\hat{k}_n, s_n \rangle s_n + w_n \partial\text{out}, \quad \partial\hat{k}_n = \partial\ell_n \cdot q, \quad r = (\frac{1}{d} \|s_n\|^2 + \varepsilon)^{-1/2},$$

with query and norm-weight gradients accumulated in registers and reduced by a deterministic two-stage tree. Because every source feeds every later routing call, a reference backward fans these contributions in through thousands of small tensor additions; the fused backward instead accumulates them into one shared fp32 buffer *in place*. This is safe because autograd executes the routing calls’ backwards in strictly reverse forward order—every later call is a graph descendant of the current call’s output through the residual chain—so when call s runs, rows $[0, N_s)$ already contain all contributions from calls $s' > s$; the first backward (the final routing) stores directly, and the ordering is asserted at runtime. Only the row owned by the call’s own residual input is handed back to autograd.

Delta variant [Luo et al., 2026]. The mid-training configuration (§3.3) routes over a short list—a learnable null source plus at most `num_blocks` block-aggregated deltas—so cross-call buffer sharing buys little. There we use a self-contained per-call function (per-call stack, per-call gradients, the kernel adds the residual stream), which composes with FSDP and activation checkpointing by construction. At the 8B configuration ($d=4096$, $H=8$, $N=9$, per-GPU microbatch 2×4096 , per-call checkpointing) routing drops from 651 ms (`torch.compile`) to 323 ms per microbatch ($2.0\times$; $6.4\times$ over the eager reference), raising end-to-end mid-training throughput by $\sim 30\%$ on $8\times\text{H100}$. The Table 5 from-scratch columns use the Table 1 depth/width at reduced batch so the eager reference also fits in memory (100M: $B=2$, 350M/1B: $B=1$; ratios are batch-invariant); the eager reference kernels are a further $1.5\text{--}2.5\times$ slower than `torch.compile` ($7.9\text{--}11.5\times$ slower than fused).

Verification. In fp32 the fused path matches the reference exactly up to reduction order: on routing chains and full models (with and without activation checkpointing), losses are identical and every parameter gradient agrees to $\leq 2.5 \times 10^{-6}$ maximum relative error. In bf16 the two paths round differently; measured against fp32 ground truth, the fused gradients are on average *closer* to the truth than the reference’s (gradient-error ratio geometric mean ≈ 0.5 across batches; gate-scalar gradients are cancellation-dominated in both paths). Outputs are bitwise identical across repeated runs. Measured cost across scales is reported in Appendix F (Table 8).

Pseudocode. Figures 9 and 10 give device-level pseudocode for the two kernels. One program (thread block) handles one position $p = (b, t)$ and holds $[H, K]$ tiles ($K=d/H$) in registers; V is the shared source buffer (rows $[0, N)$; the per-call stack in delta modes); W holds the routing weights—the only activation saved for backward. `round` marks the cast to the storage dtype at exactly the point the reference RMSNorm rounds, so the fused forward computes the same function. In the backward, dV is the shared fp32 gradient accumulator: `FIRST_CALL` (the final routing call, whose backward autograd executes first) stores, every earlier call accumulates in place; in delta modes each call owns a fresh dV and always stores. Query/norm-weight gradients leave the kernel as per-position partials and are summed on the host—a deterministic two-stage reduction, no atomics.

F Compute and memory cost

Table 8 reports training throughput and peak memory at the three model settings of Table 1 on a single H100, for three implementations of the routing. Three points. First, **single-head attention residuals and MHAR have identical cost**: the multi-head split is a parameter-, FLOP-, and memory-free reshape (§2), so single-head and MHAR share every row of Table 8 by construction ($H=1$ multi-head routing reproduces single-head exactly); any overhead is inherited from attention

residuals, not introduced by the multi-head split. Second, the overhead relative to the additive baseline comes entirely from the attention-residual *mechanism*: every sublayer routes over the whole list of up to $2L+1$ past sublayer outputs, so a training step moves $O(L^2 B T d)$ of source data, and the reference kernels pay this several times over—per-sublayer stacking of the source list, a materialized normalized key tensor, separate softmax/mix kernels, and per-sublayer retention of the stacked sources for the backward pass, which is what exceeds 80 GB at 1B. `torch.compile` fuses the arithmetic but not the stacking, retention, or backward accumulation. Third, our **fused Triton implementation** (design in Appendix E) removes the artifact, yielding $1.7\text{--}2.4\times$ the `torch.compile` path— $0.88\times/0.71\times/0.55\times$ of baseline throughput at 100M/350M/1B—at approximately baseline peak memory, with training unchanged (fp32 routing gradients match the reference to $\leq 2.5 \times 10^{-6}$ relative; bf16 within rounding noise; loss curves coincide). The remaining gap to baseline is the $O(L^2 B T d)$ source re-read inherent to dense depth routing, which the fused kernels run at roughly HBM bandwidth.

Table 8: Training compute and memory at the three model settings of Table 1 (single H100; per-GPU microbatch as in training: batch 8 / sequence 2048 at 100M, 4/1024 at 350M, 2/1024 at 1B; identical node and software for all cells; 120-step runs on a verified-healthy GPU). Each cell is the median of three steady-state windows (within $\pm 1\%$); peak memory reproduces to the decimal across sessions, and the 350M compiled/fused cells reproduce in a second independent session. Throughput is relative to the same-scale baseline (absolute baseline medians: 113.9k / 45.0k / 19.7k tokens/s). Parameters are matched to the baseline up to the $O(d)$ routing queries (+0.02%). [†]Single-head attention residuals are cost-identical to MHAR *by construction* in every row—the head split is a parameter-, FLOP-, and memory-free reshape (§2)—so they are not listed separately.

Method [†]	100M		350M		1B	
	thr.	mem. (GB)	thr.	mem. (GB)	thr.	mem. (GB)
Baseline (additive residual)	1.00×	41.5	1.00×	19.4	1.00×	19.0
MHAR, reference kernels	0.31×	68.8	0.16×	70.9	OOM (>80 GB)	
MHAR, <code>torch.compile</code>	0.54×	52.4	0.32×	40.1	0.23×	47.5
MHAR, fused Triton (ours)	0.88×	42.0	0.71×	20.0	0.55×	20.1

G Width-control: isolating width in loss

The probe (Table 6, control (ii)) isolates *width* as the driver of subspace disagreement: widening $d_{512} \rightarrow d_{768}$ at fixed depth/KV raises the learned disagreement $0.235 \rightarrow 0.281$ while the random-query null and source collinearity stay flat. Here we show the same widening also moves the *loss*, closing the link from disagreement to cost in-distribution (unlike an inference-time intervention, which is off-distribution). We train all three methods at the wider $d_{768}/L_{12}/kv_4$ point (“124M”) on the web corpus of the replication (FineWeb-Edu; Table 10) under a unified 5×10^{-4} recipe (20K steps, sequence 2048, global batch 64, $L=12$, $kv=4$) against a matched d_{512} baseline trained the same way (the 100M *architecture* of Table 1; only d , the head count, and the FFN width change). This control is a self-contained 5×10^{-4} comparison on the web corpus, so its d_{512} deltas differ slightly from Table 10’s tuned-rate (1×10^{-3}) numbers. Within each width all three methods share the same data-parallel degree, so the method-vs-baseline *deltas* cancel world-size effects and are directly comparable across widths (Table 9).

H Robustness of the from-scratch comparison

Every run in this appendix trains on web-generic FineWeb-Edu [Penedo et al., 2024] with randomly initialized routing queries (predating the zero-initialization fix; §3), as do the head grids of Appendix C and the checkpoints probed in Appendix D.

Web-data comparison (FineWeb-Edu). Table 10 repeats the four-method comparison of Table 1 on web-generic FineWeb-Edu [Penedo et al., 2024] at the tuned peak learning rate 1×10^{-3} , with the compute-equivalent gain of MHAR (see §3). Two differences from the main-text protocol: these runs predate the zero-initialization fix (routing queries randomly initialized, deviating from Kimi

Table 9: Width-control (FineWeb-Edu, tail-mean validation loss, steps 15–20k). Holding depth ($L=12$) and KV (4) fixed and only widening $d_{512} \rightarrow d_{768}$: single-head routing’s edge over the baseline *erodes* ($-0.041 \rightarrow -0.033$) while MHAR holds ($-0.050 \rightarrow -0.053$), so MHAR’s advantage over single-head *more than doubles* ($-0.009 \rightarrow -0.020$). This is the loss-level counterpart of the width-isolated disagreement in Table 6 (which rises $0.235 \rightarrow 0.281$ over the same widening). Single-seed; the single-head deficit shift (0.008) is near the per-seed noise floor, so the robust signal is the widening MHAR—single gap.

Width	Baseline	Single-head	MHAR	MHAR—single
100M (d_{512})	3.462	3.421 (-0.041)	3.412 (-0.050)	-0.009
124M (d_{768})	3.220	3.187 (-0.033)	3.167 (-0.053)	-0.020

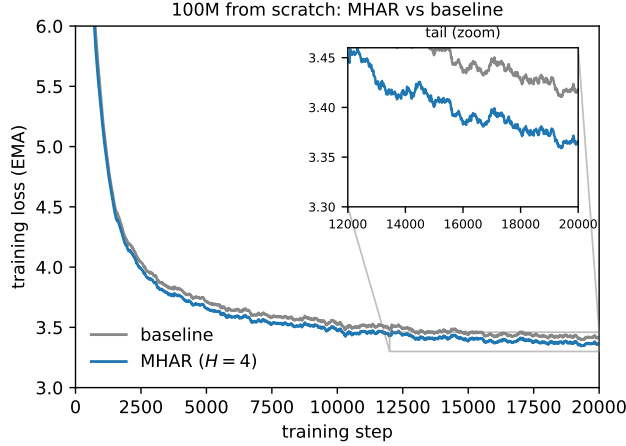


Figure 11: Training loss at 100M on FineWeb-Edu (EMA-smoothed): MHAR stays below the standard baseline throughout training (inset: tail zoom, steps 12k–20k). The two runs are identical except for the routing mechanism (same node, software, data order, and global batch).

[2025]; on this corpus zero-init moves the 1B single-head delta from $+0.082$ to $+0.018$ at 5×10^{-4} but does not change its sign at the tuned rate), and each method is shown at its tuned rate. The single-head column reverses relative to the anneal corpus: helpful at 100M, above baseline at 350M–1B, while MHAR keeps its full gain—the distribution-dependence discussed in §3.

Table 10: Web-data (FineWeb-Edu) validation loss ($\downarrow$, tail-mean) at the tuned peak learning rate 1×10^{-3} ; Δ vs. the same-rate baseline; $\text{CEG}_{\text{FLOPs}}$ is the compute-equivalent gain of MHAR over baseline. Randomly initialized routing queries (pre-fix; see text).

Scale	$d/L/KV$	Baseline	Hyper-conn.	Single-head	MHAR	$\text{CEG}_{\text{FLOPs}}$
100M	512/12/4	3.336	3.322 (-0.014)	3.297 (-0.039)	3.287 (-0.049)	$1.27\times$
350M	1024/24/8	3.290	3.260 (-0.030)	3.345 ($+0.055$)	3.210 (-0.080)	$1.49\times$
1B	1280/36/8	3.175	3.172 (-0.002)	3.315 ($+0.140$)	3.111 (-0.063)	$1.38\times$

Robustness to learning rate (best-vs-best). Fixing all methods at 1×10^{-3} is the optimum for baseline, hyper-connections, and MHAR but *not* for single-head routing, whose single shared query is destabilized by the higher rate. To rule out that this either penalizes single-head or favors MHAR, we sweep the peak learning rate over $\{1 \times 10^{-4}, 5 \times 10^{-4}, 1 \times 10^{-3}\}$ *independently per method* and compare each at its *own* best (Table 11). The lowest rate is uniformly too low (validation loss 4–5: severely under-trained at 20K steps); baseline, hyper-connections, and MHAR prefer 1×10^{-3} ; single-head prefers 5×10^{-4} at 350M and 1B (and 1×10^{-3} at 100M). Even best-vs-best, MHAR improves over the baseline at every scale ($-0.049/-0.080/-0.063$), and single-head—now at its own kinder 5×10^{-4} —still lands above the baseline at the larger scales ($+0.038$ at 350M, $+0.105$

at 1B, versus $+0.055/+0.140$ at 1×10^{-3}). The advantage is a property of the architecture, not of a favorable or under-tuned learning rate.

Table 11: Best-vs-best learning rate (FineWeb-Edu): each method at its *own* optimum over peak LR $\in \{1, 5\} \times 10^{-4}$, 10^{-3} (validation loss $\downarrow$, tail-mean of the last eleven evaluations; Δ vs. the same-scale best baseline; superscript is the selected LR). MHAR wins at every scale even when every method is given its own optimal learning rate; single-head, even at its kinder 5×10^{-4} , still sits above the baseline at 350M and 1B.

Scale	Baseline	Hyper-conn.	Single-head	MHAR
100M	$3.336^{10^{-3}}$	$3.322^{10^{-3}}$ (−0.014)	$3.297^{10^{-3}}$ (−0.039)	$3.287^{10^{-3}}$ (−0.049)
350M	$3.290^{10^{-3}}$	$3.260^{10^{-3}}$ (−0.030)	3.328^{5e-4} (+0.038)	$3.210^{10^{-3}}$ (−0.080)
1B	$3.175^{10^{-3}}$	$3.172^{10^{-3}}$ (−0.002)	3.279^{5e-4} (+0.105)	$3.111^{10^{-3}}$ (−0.063)

Robustness to training budget. At the fixed 20K-step budget the models are under-trained, which raises the question of whether MHAR’s advantage is merely an under-training artifact. We therefore train baseline, single-head, and MHAR for $3\times$ longer (60K steps) at 350M and 1B and read the baseline-vs-method gap at both budgets (within-run; the longer cosine schedule means the 20K read here is not comparable to Table 1, only the gap-vs-budget trend is). Tripling the budget lowers validation loss by ~ 0.25 —confirming the runs are genuinely under-trained—but does *not* erase the gaps: MHAR stays -0.056 (350M) / -0.062 (1B) ahead of the baseline at 60K (vs. -0.069 / -0.068 at 20K), and single-head routing’s 1B regression persists ($+0.018$ at both budgets). The architecture effects are thus properties of the model, not transients of a short schedule.

Seed robustness (all scales). We train *three seeds per method at every scale* under a single shared 5×10^{-4} recipe, so that the three methods differ *only* in seed and routing mechanism. Because seed s fixes the data order across methods, each method-vs-baseline comparison is *paired*, cancelling cross-seed noise. MHAR improves over the baseline at all three scales, with the paired delta exceeding ten standard errors everywhere ($-0.045/-0.090/-0.071$ nats at 100M/350M/1B; Table 12). The single-head and hyper-connection patterns also hold under seeds. Single-head routing moves from a significant *help* at 100M (-0.043) through -0.005 at 350M to a significant *regression* at 1B ($+0.093$), and is by far the highest-variance method: its three 1B paired deltas span $+0.04$ to $+0.16$, versus MHAR’s ± 0.002 —exactly what the forced-compromise account predicts for an unstable single shared query. Hyper-connections help at 100M/350M but fall within noise at 1B (-0.016).

Table 12: Seed robustness (FineWeb-Edu): paired per-seed delta vs. baseline (validation loss $\downarrow$, tail-mean of the last eleven evaluations; three seeds per method per scale, shared 5×10^{-4} recipe; $\pm$ is the per-seed standard deviation). Seed s fixes the data order across methods, so the comparison is paired. MHAR improves at every scale with $|\Delta|/\text{SE} \geq 10$; single-head crosses from help to harm and is the highest-variance method.

Scale	Hyper-conn.	Single-head	MHAR
100M	-0.031 ± 0.003	-0.043 ± 0.003	-0.045 ± 0.007
350M	-0.028 ± 0.009	-0.005 ± 0.015	-0.090 ± 0.004
1B	-0.016 ± 0.015	$+0.093 \pm 0.049$	-0.071 ± 0.002

I Training and reproducibility details

All runs share a single code path; the settings below are common to every method and scale unless noted.

Mid-training (8B). The base checkpoint is released at a constant peak rate of 1.7×10^{-3} (published before cooldown); our control’s peak 4×10^{-4} is this plateau rate scaled down by $\sqrt{\text{batch ratio}}$. Because no 8B plateau checkpoint ships loadable optimizer moments, we cold-start AdamW [Loshchilov and Hutter, 2019] and use a short linear re-warmup ($0 \rightarrow \text{peak}$) to rebuild the

second moment before decaying. Training uses FSDP full-shard, bfloat16, and activation checkpointing on 8×H100-80GB, sequence length 4096, weight-EMA ($\beta=0.999$, evaluated), a logit z-loss (10^{-4}), and weight decay 0.05 (excluding embeddings, norms, biases).

Tokenizer and data. We tokenize the `anneal_pt_v3` corpus (§3.3; composition in Table 13) with the Qwen3 byte-level BPE tokenizer (loaded from Qwen/Qwen3-0.6B). Documents are streamed from the corpus shards, joined with the end-of-text token (id 151,645), and packed into contiguous sequences of length T with no padding. The token-embedding and output projection are tied (`tie_word_embeddings`) with model vocabulary size 151,936. The supplementary web replication (Appendix H) runs the identical pipeline on FineWeb-Edu [Penedo et al., 2024].

Architecture. Every model follows the Qwen3 recipe [Qwen, 2025]: pre-norm decoder blocks with RMSNorm ($\epsilon=10^{-6}$), SwiGLU (SiLU) MLPs, grouped-query attention with per-head query/key RMSNorm, and rotary position embeddings (RoPE, $\theta=10^4$, the training-code default; no learned or absolute positional embeddings), with `max_position_embeddings=2T`. Attention biases are disabled and weights are initialized from $\mathcal{N}(0, 0.02^2)$. There is no dropout (attention and hidden dropout are 0).

Optimization. AdamW [Loshchilov and Hutter, 2019] with $\beta_1=0.9$, $\beta_2=0.95$, $\epsilon=10^{-8}$, and weight decay 0.1 applied uniformly to all parameters (no no-decay group). Gradients are clipped to global norm 1.0. The learning rate warms up linearly over 1,000 steps to the peak and then follows a cosine decay to $0.1 \times \text{peak}$ (the code’s default `lr_min` ratio) over the full 20K-step budget. Training uses bfloat16 (parameters and compute) under distributed data parallelism at all scales up to 1B.

Batch and sequence length. The 100M models use sequence length 2048 and global batch 64; the 350M and 1B models use sequence length 1024 and global batch 32. The base random seed is 42 (offset per worker).

Validation protocol. There is no explicit held-out split: validation draws from the *same* corpus under a different shuffle order of the stream (a +9999 seed offset from training, plus a per-worker offset), which reduces rather than eliminates train/validation overlap. Every 500 steps we run one evaluation pass: the token-level cross-entropy is averaged over the pass’s evaluation batches and then averaged (all-reduce mean) across data-parallel workers. Reported numbers are the *tail-mean* of the last eleven evaluations (steps 15–20K); the single-pass ± 0.07 noise and the paired-comparison rationale are discussed in §3 (Metric).

Mid-training corpus. Table 13 gives the measured composition of the `anneal_pt_v3` corpus used for the 8B mid-training experiments (§3.3). The corpus is assembled from public, quality-filtered corpora, shuffled at the document level into 2,048 uniform parquet shards (≈ 8 TB of text, ≈ 1.9 T tokens); every document carries a `source` tag and its upstream quality metadata, so shares are measured, not estimated.

Table 13: Composition of the anneal_pt_v3 mid-training corpus: share of text bytes per source, measured over a uniform random sample of 48 of the 2,048 shards (187 GB of text; group rows sum their constituents, so rounded columns may differ in the last digit). The corpus is English-only; FinePDFs is restricted to its top three of twenty quality bins (verified from the per-document final_bucket metadata).

Source (per-document source tag)	% bytes
<i>Synthetic web rephrasings (Nemotron-CC-v2/v2.1)</i>	24.3
nemotron-cc-v2-high-quality-synthetic	14.9
nemotron-cc-v2.1-high-quality-synthetic	6.0
nemotron-cc-v2.1-high-quality-translated-to-english-synthetic	3.5
<i>Raw high-quality web (Nemotron-CC-v2/v2.1)</i>	16.6
nemotron-cc-v2-high-quality	12.5
nemotron-cc-v2.1-high-quality-translated-to-english	2.4
nemotron-cc-v2.1-high-quality	1.6
<i>Web PDFs</i>	15.9
finepdfs (top 3/20 quality bins)	15.9
<i>Synthetic diverse QA (Nemotron-CC-v2/v2.1)</i>	14.1
nemotron-cc-v2-diverse-qa	8.3
nemotron-cc-v2-translated-diverse-qa	5.3
nemotron-cc-v2.1-high-quality-dqa	0.4
<i>Code</i>	11.9
dolma-stack-edu	6.7
nemotron-pretraining-sft-code	2.8
nemotron-cc-code-v1	2.0
nemotron-pretraining-v1.1-code-concepts	0.4
nemotron-pretraining-scientific-coding	0.1
<i>SFT / STEM / reasoning</i>	9.7
nemotron-pretraining-sft-general	4.6
nemotron-pretraining-stem-sft	4.1
nemotron-pretraining-rqa	0.7
nemotron-pretraining-infinibyte-reasoning	0.2
nemotron-pretraining-v1.1-multiple-choice	0.1
nemotron-pretraining-v1.1-formal-logic	<0.1
nemotron-pretraining-v1.1-unconditional-algorithmic	<0.1
<i>Math / arXiv</i>	6.9
nemotron-math-3	2.5
nemotron-math-4plus	1.6
nemotron-math-4plus-mind	1.1
dolma-rpj-proofpile-arxiv	0.9
nemotron-pretraining-math-textbooks	0.8
nemotron-pretraining-sft-math	0.1
<i>Wikipedia</i>	0.6
nemotron-pretraining-wiki-rewrite	0.3
finewiki-en	0.3